

InstructRepair: Instruct Large Language Models With Rich Bug Information for Automated Program Repair

Anmin Fu[✉], *Member, IEEE*, Pengyu Xu[✉], Jichunyang Li[✉], Boyu Kuang[✉],
and Yansong Gao[✉], *Senior Member, IEEE*

Abstract—Automated Program Repair (APR) repairs software bugs based on buggy code snippets automatically. It is instrumental in reducing the time and effort required for software maintenance. Recently, large language models (LLMs) have been utilized for APR and demonstrated promising results. Existing APR approaches adopt an LLM learning paradigm of either prompt engineering or fine-tuning to perform APR tasks. However, these prompt-based approaches underutilize the extensive high-quality bug-fix datasets available in the APR community, missing out on valuable repair knowledge. Conversely, fine-tuning-based APRs rely on LLMs to autonomously understand the entire complex APR tasks without the benefit of natural language guidance through a prompt. Additionally, the crucial bug information the bug-triggering test suites provided remains largely unexplored. Furthermore, while existing APRs focus only on repair effectiveness, they neglect the patch ranking effectiveness that also matters in real-world program repair scenarios. To address the above limitations, we propose InstructRepair, an innovative APR framework that integrates self-supervised instruction tuning with rich bug information to guide LLMs in generating high-quality patches. InstructRepair instructs APR tasks through detailed instructions and enhances repair performance by training LLMs over the first instructional APR dataset we construct. InstructRepair also extracts rich bug information (e.g., buggy line, bug’s context, bug’s diagnostics, test suites and meta-information) to provide fix-relevant tokens and insights into the root causes of bugs for LLMs during both instruction tuning and the repairing process. The human domain knowledge initiated instruction template that integrates this bug information is refined through prompt tuning, which learns task-specific knowledge and unleashes the hidden power stored in pre-trained LLMs automatically. We evaluate InstructRepair on the widely adopted Defects4J v1.2 and v2.0 benchmarks with

Java programming language, demonstrating that our work outperforms seven state-of-the-art APR approaches by successfully fixing a total of 122 bugs (10 more than the best baseline) and repairing 19 unique bugs that previous work cannot. In terms of patch ranking effectiveness, InstructRepair also achieves the best performance with a fixing rate of 95.9% (60.0% as the best baseline). The ablation study further validates the contributions of each component to overall repairing performance. We also apply InstructRepair to the Python programming language and beat four state-of-the-art APR approaches, which demonstrates our method’s cross-language generalization capability.

Index Terms—Automated program repair, large language model, LLM4SE.

I. INTRODUCTION

AUTOMATED Program Repair (APR) [1], [2], [3] aims to automatically fix bugs in programs without human intervention, which is crucial for software developers given the inevitability of bugs [4] and the time-consuming nature of manual bug fixing [5]. However, traditional APR methods [6], [7] rely on predefined, hand-crafted fix templates [8], limiting their effectiveness and generalizability. With the rapid development of Deep Learning (DL) techniques [9], learning-based APR approaches [10], [11] have emerged, addressing existing limitations. Among these, large language model (LLM) based APRs have achieved state-of-the-art (SOTA) performance, offering unique fixes that prior APRs cannot predict [3], [12]. This success is attributed to the stunning capability provided by the large-scale LLMs in terms of huge parameters, which are pre-trained on billions of code snippets [13], enabling them to excel in a wide range of code-related tasks [14].

Recent LLM-based APR approaches typically leverage two learning methods to perform APR tasks: prompting and fine-tuning [15]. Prompt-based APRs design prompt templates that provide buggy programs and bug information (such as the bug’s location) to LLMs [16], [17]. For instance, ChatRepair [18] employs a conversational prompt format, querying ChatGPT with buggy code and a natural language description of the APR tasks. AlphaRepair [19] proposes programming language prompts (i.e., code prompts) that consist of buggy code context and masked tokens to guide CodeBERT in filling in the correct code at the bug location. In contrast, fine-tuning-based APRs use supervised or self-supervised training on datasets that have bug-fix pairs [20] to teach LLMs how to transform buggy

Received 20 March 2025; revised 9 August 2025 and 3 September 2025; accepted 30 September 2025. Date of publication 6 October 2025; date of current version 16 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372236 and Grant 62402223 and in part by the Qing Lan Project of Jiangsu Province. The associate editor coordinating the review of this article and approving it for publication was Prof. Fengwei Zhang. (*Corresponding author: Yansong Gao.*)

Anmin Fu is with the School of Cyber Science and Engineering and the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: fuam@njust.edu.cn).

Pengyu Xu and Boyu Kuang are with the School of Cyber Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: xupengyu@njust.edu.cn; kuang@njust.edu.cn).

Jichunyang Li and Yansong Gao are with the School of Computer Science and Software Engineering, The University of Western Australia, Perth, WA 6009, Australia (e-mail: jichunyang.li@uwa.edu.au; garrison.gao@uwa.edu.au).

Digital Object Identifier 10.1109/TIFS.2025.3618407

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on November 20, 2025 at 06:04:22 UTC from IEEE Xplore. Restrictions apply.

code into a correct patch. For example, RewardRepair [21] treats buggy code as training input and human-written patch as training output, encouraging LLM to learn bug-fixing patterns.

While LLM-based APR approaches have achieved impressive results, three notable limitations remain:

① *Imbalance between APR tasks clarity and bug-fix datasets utilization.* On one hand, prompt-based APRs rely on natural language prompts and code prompts to convey APR tasks to LLMs [22]. These prompts help LLMs improve APR task clarity and can activate the LLM’s pre-trained skills effectively, as well-crafted prompts significantly improve model performance across various tasks [23]. However, existing fine-tuning-based APRs overlook the potential of prompts, expecting LLMs to automatically understand the entire complex APR tasks without any additional explicit guidance. They focus only on training LLMs with bug-fix pairs through different fine-tuning techniques without clarifying APR tasks via prompt as instructions. On the other hand, fine-tuning-based APRs leverage extensive bug-fix datasets to teach LLMs a wide variety of bug-fixing patterns. Yet, current prompt-based APRs underutilize these bug-fix data, either referring to only a few samples or operating in a zero-shot setting [19], [24], not harnessing rich knowledge provided by these available high-quality bug-fix datasets. They focus only on directly using the LLMs with zero-shot or few-shot prompting without any further training. To sum up, both prompt-based and fine-tuning-based APRs struggle to balance APR task clarity and bug-fix dataset utilization.

② *Missing test suite code.* Existing APRs typically rely on buggy line, bug’s context, and bug’s diagnostics to fix bugs, but overlook valuable information from bug-triggering test suite code. In practice, software developers frequently refer to test suite code to understand program’s expected behavior and pinpoint the root causes of bugs [25], [26]. Also, the test suite code contains valuable tokens that LLMs can leverage to predict more accurate patches. While prior work [27] simply integrates the test suite name into bugs’ diagnostics, we find that the rich test suite code can further benefit patch generation. Figure 1 presents a bug from Defects4J [28] that prior work cannot fix using only the buggy line, the bug’s context, and the bug’s diagnostics. This bug is fixed by adding a “*script*” argument, and is hard to fix since such a *script* token is nowhere to be found within the available bug’s context or diagnostics. While its test suite code reveals how the bug is produced and provides the crucial *script* token, such valuable bug information is not yet considered from current APRs.

③ *Neglecting patch ranking effectiveness.* The beam search algorithm is commonly used in APR to select only a pre-determined number (i.e., beam width) of the most promising candidate patches rather than exploring all possibilities like brute-force search. Recent APRs tend to use a large beam width (up to 5000 [29]) to produce a huge group of candidate patches for a single bug, which naturally boosts the repair effectiveness. By contrast, however, a smaller beam width is particularly advantageous when computational resources are limited or when urgent repairs [30] (e.g., 0-day vulnerability fixes) are required [2]. Also, in practical terms [31], smaller beam widths can also streamline decision-making by reducing

Bug’s Diagnostics: org.jsoup.parser.ParserTest::handlesDataOnlyTags
--> junit.framework.AssertionFailedError: expected:<Hello [There]> but was:<Hello [!]>; i++; [There]
Buggy Code:

```
Token.StartTag startTag = t.asStartTag();
String name = startTag.name();
if (name.equals("html")) {
    ...
    - } else if (StringUtil.in(name, "base", "basefont", "bgsound", "command",
      "link", "meta", "noframes", "style", "title")) {
    + } else if (StringUtil.in(name, "base", "basefont", "bgsound", "command",
      "link", "meta", "noframes", "style", "title", "script")) {
        return tb.process(t, InHead);
    }
}
```

Test suite code:

```
@Test public void handlesDataOnlyTags() {
    String s = "<p>Hello</p><script>obj.insert('<a rel=\"none\" />');\n\ni++;</script><p>There</p>";
    Document doc = Jsoup.parse(s);
    ❶ assertEquals("Hello There", doc.text());
}
```

Fig. 1. An example of test suite code helped repair.

the number of candidate patches that developers need to review manually, thereby rendering the repair process more efficient and practical for real-world workflows [32]. But, recent APR approaches neglect patch ranking effectiveness, maintaining a fixing rate of only 25.7 ~ 60.0% under a small beam width.

In this paper, we devise a novel APR framework named InstructRepair to address the aforementioned limitations and generate more accurate patches. The key insight is the introduction of a new learning strategy to APR, namely *instruction tuning* [33] unique to LLMs, such as GPT-4 [34], Llama-3 [35], and CodeT5+ [36], which can improve LLM models’ performance on various tasks (e.g., natural language generation and code generation). The application of instruction tuning in APR is underexplored because of data limitations. Previous work has mainly focused on scraping bug-fix commits from online sources and covering sufficient bug types [11], [37]. However, these datasets are not compatible with instruction tuning, as they lack clear repair instructions and mix rich bug information with buggy code. The APR community requires a new dataset that not only contains detailed instructions for APR tasks, bug-fix pairs, and rich bug information, but also explicitly organizes these elements. Our work investigates the high-quality bug-fix dataset from prior APR work [27], extracts the buggy code and rich bug information, designs an instruction template, and integrates them into the first instructional APR dataset. We further refine the instruction template we manually design upon domain knowledge into an optimal template automatically by utilizing prompt tuning [38] to integrate task-specific representations and better unleash the hidden power stored in pre-trained LLMs. Through conducting instruction tuning upon this dataset, we improve CodeT5+ to the capability of understanding diverse program repair tasks and generating high-quality fixes. Notably, InstructRepair is the first unite APR framework that can utilize strengths of the two major optimization techniques (i.e., prompting and fine-tuning) simultaneously in APR. Additionally, InstructRepair is the first to incorporate test suite code directly into the repair

process. Unlike prior methods, which use test suites only to verify the correctness of a patch *after* repairing process, we extract bug information from bug-triggering test suite code *in* repairing process along with the buggy line, bug's context and bug's diagnostics to provide CodeT5+ with deeper insights into bugs.

We evaluate InstructRepair over two widely used Java benchmarks, Defects4J v1.2 and Defects4J v2.0 [28], and compare InstructRepair with seven SOTA APR approaches, including both conventional learning-based [11], [37] and LLM-based APRs [19], [21], [27], [29], [39]. We use perfect fault localization (PFL) [40] - the exact fixed location of the bug is provided - per these APRs to avoid potential deviations that fault localization may bring about and ensure a fair comparison. The results show that InstructRepair outperforms all SOTA methods. In total, InstructRepair successfully repairs 69 and 53 bugs from Defects4J v1.2 and v2.0, respectively, and generates 19 unique patches that prior work cannot produce. Among these SOTAs, InstructRepair achieves the best performance on patch ranking effectiveness, exhibiting 95.9% of the repair ability when operating with a small beam width. Besides, we examine the contributions of each component in InstructRepair, demonstrating that the innovative combination of instruction tuning and rich bug information improves repair performance complementarily. Finally, we extend InstructRepair to another popular Python programming language to evaluate its cross-language generalization capability. InstructRepair achieves the new SOTA record of successfully repairing 30 bugs (15% more than the best baseline) from BugsInPy [41], which is a widely adopted real-world open-source APR benchmark.

To sum up, this paper makes the following contributions:

- We bridge the gap between two major techniques in LLM-based APRs: prompting and fine-tuning. Our work proposes and constructs the first instructional APR dataset to deal with the data limitations issue and explores instruction-tuning in the field of APR for the first time. We are also the first to incorporate bug-triggering test suite code into the repair process to exploit rich bug information. This paper also pinpoints the patch ranking issue of APRs and analyzes existing APR approaches in practical terms.
- We propose InstructRepair, a unite APR framework that can effectively align LLMs with APR tasks by complementarily leveraging two LLM learning paradigms of prompting and fine-tuning. By providing explicit instructions, leveraging bug-fix datasets and exploiting rich bug information from the buggy line, bug's context, bug's diagnostics, bug-triggering test suites and meta-information, InstructRepair improves LLMs' both repair and patch ranking effectiveness. We also integrate task-specific representations in human domain knowledge-initiated instructions and refine them into an optimal instruction template automatically through prompt tuning, further boosting InstructRepair's performance.
- We conduct extensive experiments on two widely adopted APR benchmarks, Defects4J v1.2 and v2.0 for Java, and

compare InstructRepair against seven SOTAs. The experimental results show that InstructRepair repairs a total of 122 bugs (10 more than the best baseline) and provides 19 unique patches, demonstrating the repair effectiveness of InstructRepair. We examine InstructRepair's patch ranking effectiveness under different beam widths, indicating that our work remains the most effective (95.9%) in practical terms compared with SOTAs (60.0% as the best baseline). Additionally, we perform an ablation study to justify the contribution of individual components within InstructRepair. Finally, we implement our tool on another popular Python programming language to demonstrate InstructRepair's cross-language generalization capability. InstructRepair beats four Python SOTAs by generating 15% more patches from BugsInPy.

- We have open-sourced the first instructional APR dataset, InstructRepair's code, experimental results and processed data to support the open science community¹.

II. BACKGROUND AND RELATED WORK

A. Automated Program Repair

Software bugs can cause severe consequences, disrupting businesses [42] or even governments [43]. APR [1], [2], [3] is designed to mitigate this issue by automatically fixing bugs, thereby easing the burden on software development and operations teams [44].

Traditional APR approaches can be categorized into three types: search-based [45], constraint-based [46], and template-based [47]. Search-based APR approaches first establish search spaces that contain potential patches by transforming the buggy code or gathering the correct code snippets from available repositories. Then, they develop search strategies based on biological engineering [48], [49] or code similarity [50] to explore the search spaces and recommend repair suggestions. Constraint-based APR approaches develop constraint rules from test cases or static analysis techniques [46], [51]. By performing constraint solving, they prune the search spaces and generate suggested patches. Template-based APR approaches utilize hand-crafted fix patterns [47], which denote the general code change in real-world program repair process [8], [52], to produce candidate patches. However, traditional APR approaches fail to learn high-level repair knowledge from available high-quality bug-fix datasets automatically, and still frequently generate low-quality uncompileable patches [2], [15].

With the emergence of deep learning technologies, learning-based APRs properly handle these issues with the help of neural network models [11], [37], [53]. Most recently, the rapid advancement of Large Language Models [14] has enabled researchers to explore their potential in various natural language processing (NLP) tasks and software engineering tasks [54], [55], including APR. Specifically, LLM-based APR approaches leverage two optimization techniques to perform APR tasks: prompting [56] and fine-tuning [57].

Prompting is a practical technique to interact with LLMs. A prompt contains the description of tasks that users expect

¹The artifacts are available at <https://github.com/FzBacon/InstructRepair>

LLMs to conduct and may include other details to support LLMs in properly producing results. Prompt-based APR approaches directly leverage the pre-trained language processing capabilities of LLMs. Some of them view APR tasks as cloze tasks [19], [29] where buggy code is replaced with blanks (i.e., masks). The mask strategy is inspired by real-world bug-fix patterns. Through the mask process, prompting-based APRs obtain a set of differently masked buggy codes that contain plausible fix patterns. By querying LLMs to fill correct code snippets in the masked position, they get a group of candidate patches. Besides, other prompting-based APRs repair programs as a conversation. They use natural language prompts [58], [59], [60] to interact with LLMs and programming language prompts to provide buggy code.

AlphaRepair [19] is the first to adopt prompting LLMs to APR, using a set of manually designed repair templates and sequentially prompting the LLM with each one. Following this, several other approaches have emerged that leverage prompting. For instance, Gamma [61] and TYPEFIX [62] have extended AlphaRepair by refining prompt templates. They propose new template selection algorithms to pair the abstract syntax tree (AST) of template code with buggy code. Repilot [29] prompts LLMs to generate patches one token at a time. But for each token, Repilot implements a completion engine to prune away the infeasible tokens and only keep those statically valid tokens, which results in more valid and compilable patches.

Fine-tuning is another widely adopted optimization technique. It is one common form of transfer learning—other forms can include knowledge distillation. In the context of this work, fine-tuning involves further tuning LLMs on specialized datasets to focus on specific downstream tasks. Fine-tuning-based APR approaches scrape the code commits from online sources (e.g., Github public repositories) and filter out those bug-fix-related commits to construct large-scale APR training datasets [11]. Most recently, self-supervised learning [63] is introduced to the APR that some fine-tuning-based APR approaches perturb buggyless code from online sources into buggy variants, constructing APR training datasets in a self-supervised manner [27]. Through these bug-fix datasets, fine-tuning-based APRs train LLMs to capture the hidden relationship between buggy code, bug’s diagnostics and correct code and learn to transform bugs directly to patches [64], [65].

CURE [37], RewardRepair [21], SelfAPR [27] and similar work [39], [66] have explored typical fine-tuning methods to enhance LLMs by training them with bug-fix datasets. CURE [37] fine-tunes the GPT model with a bug-fix dataset that is constructed by extracting bug-fix-related commits based on identifying certain keywords (e.g., “fix”, “bug”, and “patch”) in commit messages, and learns to transform buggy code directly to correct code without predefined repair templates. RewardRepair [21] proposes a discriminative model to judge the candidate patches during fine-tuning. The loss of each fine-tuning step will be increased or decreased based on the judgment. SelfAPR [27] introduces execution diagnostics to provide fix-relevant information to LLMs during fine-tuning. RAP-GEN [39] utilizes SelfAPR’s dataset to fine-tune its

patch generator and proposes a patch retriever to offer the top relevant bug-fix pair as a guiding fix pattern. This fine-tuning process helps LLMs learn both programming language structures [67] and bug-fixing patterns [68], improving their performance in APR tasks.

However, all previous work focuses on either prompting or fine-tuning independently, with little exploration of how the two techniques might complement each other. As a result, exploration in these two approaches has remained isolated from one another.

B. Instruction Tuning

Instruction tuning is a novel method proposed by Google Research [33] to enhance LLM performance on a variety of NLP tasks. The method is grounded in the idea that all NLP tasks can be described using natural language instructions. These instructions are predefined to give general descriptions of tasks and are used as templates to arrange different components of tasks as one structured input for LLMs. Instead of directly prompting LLMs with such input, instruction tuning first fine-tunes LLMs on datasets described via these instructions, which results in better inference-time text interactions across many NLP tasks.

CodeT5+ [36] is a code foundation LLM that applies instruction tuning to the code domain. It follows Code Alpaca’s [69] settings to instruction-tune the language model on code generation, code editing, and code optimization tasks, and has established new SOTA results.

We note that recent LLMs [34], [35], [70] have adopted instruction tuning for a wide range of code understanding and generation tasks. However, the programs generated by LLMs can also contain bugs as human-written programs [29] because the APR tasks are overlooked through such a tuning process. There remains a gap between LLMs and APR tasks [15], and the APR community requires an instructional dataset but does not have it yet to apply instruction tuning properly.

C. Test Suite

Manual software testing is time-consuming and costly [25]. To make sure software is as per the requirements of software designers, test suites are proposed to automatically judge software’s behavior and are highly involved in today’s software development process [71]. In terms of bug finding, test suites are widely adopted [72]. Software developers can execute test suites to find violations of the program’s expected behavior. In terms of bug fixing, consequently, test suites are also helpful for developers in practice, because the failing test suite can reproduce the bug under fix [73] and pinpoints the root cause that triggers the bug [74]. Apart from that, some keywords from test suite code can even be directly synthesized in patches, providing extra context for bug fixing as depicted in Figure 1.

However, in the field of APR, the valuable bug information within the test suite is underexplored. Some approaches only leverage buggy code and its context to predict candidate patches [11], [19], [37], while others extract failing messages from test suites to provide bug’s diagnostics [21]. Besides,

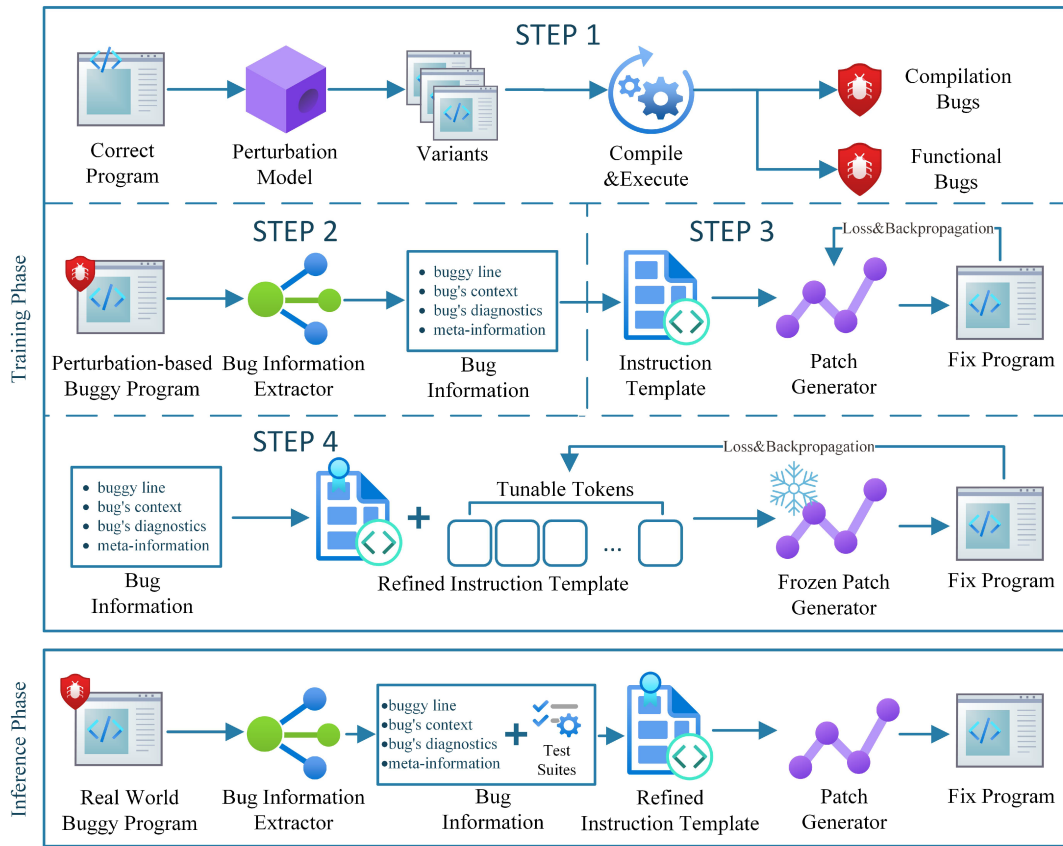


Fig. 2. An overview of InstructRepair.

recent work [27] utilizes test suite names (e.g., *handles-DataOnlyTags*, *testEquals*) to represent the general purpose of test suites. But not all of them apply test suite code during the repair process, missing additional valuable guidance on the root cause of bugs and potential fix-relevant tokens for LLMs.

III. INSTRUCTREPAIR

A. Overview

InstructRepair is a framework for APR that leverages instruction tuning with rich bug information. Figure 2 illustrates the overall design of InstructRepair, which contains two main phases: the training phase and the inference phase.

We divide the training phase of InstructRepair into 4 steps. In Step 1, InstructRepair perturbs the correct source code into its variant version to generate a large set of buggy variants for training. These variants only have slight differences from the original code but may encounter compilation failures or execution failures after being compiled and executed. InstructRepair identifies and filters out those buggy variants as perturbation-induced buggy programs for later training—note that we discard those buggyless variants because they are unrelated to APR tasks. In Step 2, InstructRepair utilizes both static and dynamic analysis to extract detailed bug information from the perturbation-induced buggy programs to provide extra guidance for repairing. In Step 3, to ensure effective learning, InstructRepair defines an instruction template that provides

descriptions of the extracted bug information. Through these structured inputs, the LLM as a patch generator is instruction-tuned to improve its repair ability. In Step 4, InstructRepair prepends additional tunable tokens to the instruction template we manually designed upon human domain knowledge and freezes the parameters of the patch generator, which allows us to further train the prompts and find a refined instruction template through prompt tuning automatically.

In the inference phase, InstructRepair handles real-world buggy programs by extracting rich bug information along with the bug-triggering test suites. To ensure clarity, extracted bug information is structured using the refined instruction template and the LLM patch generator utilizes this enriched input to generate fix programs.

B. Training of InstructRepair

Step 1: Perturbation. InstructRepair employs a perturbation model to generate buggy versions of correct programs. The perturbation model consists of 16 perturbation rules that are inspired by real-world bug-fixing patterns devised from SelfAPR [27]. Correct programs are deliberately modified by these perturbation rules to generate buggy variants. For example, the perturbation model may add an “and” logic to boolean expressions, or modify “>” operator to “≥”. Such modifications will later teach patch generators various strategies that human developers adopt in real-world bug repairing process. InstructRepair then compiles and executes

the perturbed samples and filters out the compilation bugs and functional bugs for later bug information extraction. This perturbation process can offer a substantial volume of labeled bug-fix training samples without any human intervention.

Step 2: Bug information extraction. Bug information plays a crucial role in generating accurate patches. InstructRepair first locates the buggy line and bug's context based on PFL. Then, InstructRepair collects the bug's diagnostics during the compilation or execution process as another rich bug information. These diagnostics explicitly highlight the defect type and misbehavior, offering crucial guidance for patch generators. Besides, our work extracts additional meta-information of buggy code, including class names, method names, return types and variable names through static analysis, to enhance the patch generator's structural understanding for buggy code. This extra information helps the patch generator analyze and understand buggy code more effectively.

Step 3: Training with instruction. A well-structured prompt template is essential for instructing the APR tasks and bug information to the patch generator. InstructRepair designs the first instructional prompt template for training.

Figure 3a and Figure 3b give an example of the prompt template used by previous APRs. These prompts can leverage the pre-trained language processing capabilities of LLMs by framing the APR tasks as a text-based interaction. InstructRepair takes this concept further by designing a structured instructional prompt template, as depicted in Figure 3c, to instruction-tune LLMs on APR tasks. The difference between the prompt template of previous approaches and ours (colored yellow) is that ours organizes the prompt into four distinct parts: prompt description, instruction, input, and output. Specifically, in the prompt description part, we first leverage detailed natural language prompts to clarify the overall tasks and describe the structure of the prompt. Then, in the instruction part, we summarize the general APR process and present rich bug information along with explanations to guide the patch generator. Finally, in the input part, InstructRepair offers buggy code and meta-information in programming language format, and leaves the output part empty to let the patch generator fill in the predicted patch automatically.

We fill all perturbation-induced buggy programs along with rich bug information into the instruction template to construct the first instructional APR dataset, and utilize it as the patch generator's training samples. In this way, InstructRepair ensures that the patch generator first focuses on the natural language prompt to figure out the structure of the entire prompt, and also comprehends the workflow of APR tasks along with the bug information. Then, based on the instructions, the patch generator initiates the processing of the buggy code, identifies the location of bugs, and makes use of the rich bug information to fix programs.

We choose CodeT5+ [36], which is a representative encoder-decoder based LLM, as our vanilla patch generator, because it is open-source and has demonstrated competitive performance over other code-related tasks (e.g., code defect detection, code completion) through advanced learning techniques while other LLMs [70], [75], [76] are untested under instruction tuning or prompt tuning. The encoder of CodeT5+

You are an Automated Program Repair Tool.
The following code contains a **buggy line** that has been removed.

Buggy Context
>>>[INFILL]<<<
Buggy Context

This was the original **buggy line** which was removed by the infill location:

Buggy Line

This code fails with the following **error** :

Bug 's Diagnostics

Please provide the correct line at the infill location.

(a) Natural language prompt

Comment Buggy Line
Buggy Context
<mask><mask>.....<mask><mask>
Buggy Context

(b) Code prompt

Below is an instruction that **describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.**

Instruction:

The following code contains a **buggy line** with the following **error**:

Bug 's diagnostics

The bug triggering **test suite** is:

Test Suite Code

Please provide the correct code line.

Input:

[BUGGY] *Buggy Line*
Buggy Context
[BUGGY] *Buggy Line* [BUGGY]
Buggy Context
[CLASS] *Class Name* [METHOD]...

Output:

Fix Program

(c) Instruction prompt

Fig. 3. Comparison of prompt templates in APR.

learns to encode contextual representations from code or text sequences (e.g., buggy code, bug's diagnostics). The decoder is trained to generate patches depending on the input data. We follow prior work [21], [27], [39] not to re-train the patch generator from scratch, because the pretraining of CodeT5+ involves various code-related tasks (e.g., code defect detection,

code completion), so that valuable knowledge learned through such pretraining process can be transferred to APR tasks.

Step 4: Instruction template tuning. Although we refer to Code Alpaca [69] to divide our prompt into different parts and follow prior APR work [59] to provide both hand-crafted natural language and programming language prompt, the hand-crafted instruction template may be sub-optimal considering the natural gap between APR tasks and the original NLP and code tasks LLMs are trained on. Thus, InstructRepair utilizes prompt tuning to refine the instruction template that we manually designed in Step 3 and better unleash the hidden power stored in pre-trained LLMs. Prompt tuning [38] is an effective means that can better align language models with downstream tasks by automatically tuning the model inputs. It has shown promising results in language understanding, question answering, and other NLP tasks. Accordingly, we explore the application of prompt tuning in the APR field and exploit it to refine our manually designed instruction template to find an optimal template automatically.

We follow the instruction tuning process to set the task type of prompt tuning as sequence-to-sequence tasks, which means language models will first encode the input sequence (e.g., instruction template filled with bug information) to one vector that represents the entire input, and then decode the vector to unfold it into a new sequence (e.g., fix program). And, different from instruction tuning that updates the model's parameters through backpropagation, we freeze all the parameters of LLMs during prompt tuning and prepend continuous tunable tokens (i.e., soft prompts [38]) to our instruction template. Such continuous tokens do not represent discrete words and are initialized randomly with no semantic meaning. During the prompt tuning process, language models will train the tunable tokens of the instruction template through backpropagation, so that the template is automatically refined by integrating task-specific representations.

C. Inference With Rich Bug Information

After the patch generator has been instruction-tuned, InstructRepair is fed with a real-world buggy program by first extracting detailed bug information, following the same pipeline used in the training phase. In addition, our work incorporates bug-triggering test suite code into the repair process. Note that the test suite code is not used for training but only during inference because incorporating it into the training is costly.

Specifically, we begin by identifying the test suite file and the test method that triggers the bug based on the bug's stack trace. We then parse the AST of the test suite file to isolate the relevant test method code. Next, we monitor the class loader during the execution of this test suite to pinpoint the exact line of code that directly triggers the bug. Also, with the AST of this test suite, we can precisely locate the associated context of this test line. These test suite snippets not only identify the root causes of the bug but also facilitate the generation of more accurate patches by LLMs.

Finally, we incorporate real-world buggy programs along with such extracted rich bug information into the refined

instruction template, and employ beam search with a small beam width to generate a set of high-quality candidate patches.

IV. EXPERIMENTAL METHODOLOGY

A. Research Questions

Experiments aim to answer the following research questions (RQs):

- RQ1 (Repair Effectiveness): How does InstructRepair compare against SOTAs?
- RQ2 (Patch Ranking Effectiveness): How does InstructRepair perform under different beam widths?
- RQ3 (Ablation Study): What is the contribution of each component within InstructRepair?
- RQ4 (Generalizability): How does InstructRepair perform across different programming languages or datasets?

B. Studied Dataset

For training, we utilize the dataset from SelfAPR [27] for Java, which comprises over one million Java defect samples representing a wide variety of defect types. We choose CSSB-2.6M [77] for Python, which contains over 2.6 million Python bug fixes from 500K public git repositories with 20 bug type categories. Based on these two datasets, InstructRepair extracts rich bug information from them, and integrates them to construct the training datasets that are suitable for InstructRepair as detailed in Section III-B.

For evaluation, we employ the widely adopted APR benchmark Defects4J [28] for Java. Specifically, Defects4J v1.2 is the most adopted APR evaluation benchmark [2], [3]. It includes 391 real-world bugs (removing 4 deprecated bugs) from 6 open-source GitHub projects, each of which can be reproduced and validated within a provided environment. Additionally, Defects4J v2.0 is a new version of Defects4J v1.2, which contains 438 bugs on top of Defects4J v1.2. With the development of APR, the most recent approaches [19], [21], [27], [29] start to select Defects4J v2.0 to investigate APR's generalizability and ability to fix complicated bugs [78]. For a fair comparison, we choose only the single-hunk bugs in Defects4J for evaluation per prior work [11], [19], [21], [27], [29], [37] since existing LLM-based APRs mainly focus on single-hunk fixes. We follow the exact settings of SelfAPR [27] to remove the 17 bugs that are used for constructing the dataset used for training from Defects4J to avoid the dataset-overfitting issue.

We utilize the broadly adopted APR benchmark BugsInPy [41] for the evaluation of InstructRepair over Python programming language. BugsInPy consists of 493 bugs from 17 real-world Python programs including "matplotlib", "pandas", "scrapy", and some other popular Python projects. Each of the bugs can also be reproduced and validated as Defects4J within a provided environment. Note that we filter out those single-hunk bugs as in Java, but we do not exclude any extra bugs of BugsInPy because we construct our Python training dataset based on CSSB-2.6M, which does not involve any bugs in BugsInPy.

C. Metrics

We adopt the **Identical**, **Plausible**, and **Correct** metrics which are commonly used by prior work [2], [3] to evaluate the performance of APR approaches. A patch is marked as *Identical* if it exactly matches the human-written patch which is collected from the official developer commit included in the benchmark datasets [28], [41]. But, these human-written patches are not considered uniquely correct. A patch is marked as *Plausible* if it passes not only the test case that originally exposed the bug but also all available relevant test suites to ensure it does not potentially harm any other functionality. A patch is marked as *Correct* if it is either identical or plausible and is further manually verified as functionally equivalent to the developer patch. Manual validation is conducted independently by the authors in the same testing environment used by the official developers, following the exact protocol used in baselines to ensure consistency.

D. Implementation

Our work uses CodeT5+ model from the Hugging Face API [79] and applies instruction tuning through the Transformers library [80]. We parse AST of test suites based on javalang.

In terms of hyper-parameters configuration, We follow prior work [33] to perform a few rounds of prompt tuning after instruction fine-tuning, and we also follow this baseline [27] to set tuning epochs to 10 while others' [11], [21], [37], [39] vary from 15 to 50. We align with baselines [19], [21], [39] to set the number of tokens to 512 (the rest of them [11], [27], [29], [37] set it from 768 to 1500). It's worth noting that fewer token numbers means InstructRepair needs to refer to less context to fix bugs, which is efficient and favors industrial deployment. Considering that the vanilla patch generator we chose has been pre-trained on various code understanding and generation tasks, we set a small learning rate to ensure stable optimization and preserve pre-trained knowledge. Besides, APR tasks are inherently sensitive to subtle syntax and semantic variations compared to other code generation tasks, so a small learning rate allows our model to adapt to repair signals gradually and accurately. We further leverage warm-up steps [81] to ensure stable and effective tuning. Considering the complexity of APR tasks, we set moderately larger tunable tokens to better encode task-specific semantics.

Specifically, we set the training epoch to 7, with a learning rate of 5×10^{-5} and 4000 warm-up steps for instruction tuning and set the training epoch to 3, training task to sequence-to-sequence task, number of tunable tokens to 40, with a learning rate of 3×10^{-5} for prompt tuning. We set the beam width to 20 and use ensemble models to utilize not only the final checkpoint, but also epoch checkpoints per prior work [11], [21], [27], [37], [39], [82]. All experiments are conducted on one Ubuntu 20.04.6 server with 4 Nvidia RTX A6000 GPUs, 2 Intel Xeon Platinum 8380 CPUs, 128GB memory, and 4TB SSDs. The training time of InstructRepair is 15 days, and the inferring time of it is 4 days. We also set a time-out of 5-hour for validating each bug, consistent with prior work [11], [19], [27].

E. Methodology for RQ1 & RQ2

In RQ1, we compare InstructRepair against five SOTA fine-tuning-based approaches (CoCoNuT [11], CURE [37], RewardRepair [21], SelfAPR [27], and RAP-GEN [39]) which use the same encoder-decoder architecture as InstructRepair, and two SOTA prompt-based approaches (AlphaRepair [19] and Repilot [29]) which use encoder-only and encoder-decoder architecture. Evaluations are conducted on widely studied Defects4J v1.2 and v2.0. We compare not only the quantity of bugs that APR approaches can fix, but also the number of unique bugs that one approach can fix but all other approaches cannot, to examine our work's effectiveness comprehensively.

In RQ2, we perform the evaluation of InstructRepair's patch ranking effectiveness by investigating InstructRepair's performance under different beam widths. We also compare the performance between InstructRepair and these SOTAs under a small beam width to validate our work's patch ranking effectiveness. We define the small beam width as 10 because such a beam width is indeed a relatively small beam width considering their original beam width (from 50 to 5000). In addition, a recent study recognizes that the top-10 candidate patches (i.e., candidate patches under a beam width of 10) are critical for developers' acceptance in practice [32].

F. Methodology for RQ3

In RQ3, we evaluate the contribution of different components within InstructRepair. Specifically, we perform an ablation study comparing the full system (denoted as InstructRepair_{full}) under Defects4J against 4 variants: InstructRepair without any instructions or test suite code (denoted as InstructRepair_o), InstructRepair with hand-crafted instructions (denoted as InstructRepair_{w/ instruction}), InstructRepair with test suite code (denoted as InstructRepair_{w/ test-suite}), and InstructRepair with both hand-crafted instructions and test suite code but without refined instructions (denoted as InstructRepair_{w/o prompt-tuning}). Due to the high cost of re-training InstructRepair from scratch, we set training epochs to 3 as InstructRepair_{full}, and apply the same settings to these variants. This evaluation evaluates the impact of instruction tuning and test suite code on overall repair performance.

G. Methodology for RQ4

In RQ4, we evaluate InstructRepair's performance over another broadly applied programming language, which is Python, to examine the generalizability of our work. We compare InstructRepair with four SOTAs (CoCoNuT [11], AlphaRepair [19], Pyter [7], and TypeFix [62]) that are either specially designed to repair Python bugs [7], [62] or initially implemented on Java, yet extended over Python to evaluate their cross-language repair capability [11], [19] like our work.

V. EXPERIMENTAL RESULTS

A. RQ1 (Repair Effectiveness): How Does InstructRepair Compare Against SOTAs?

In RQ1, we compare InstructRepair against SOTAs to evaluate repair effectiveness. As shown in Table I, InstructRepair

TABLE I

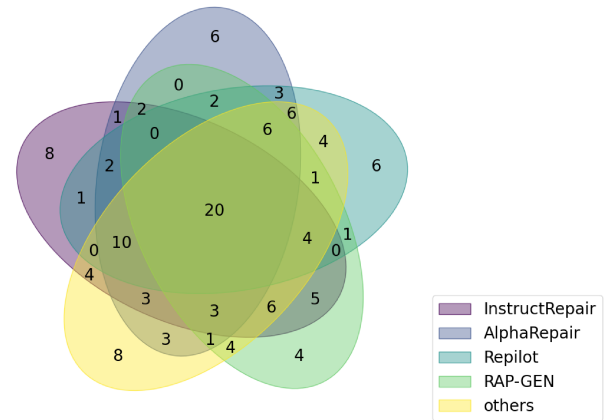
COMPARISON OF REPAIR EFFECTIVENESS WITH SOTAS OVER DEFECTS4J v1.2 AND v2.0. IN THE CELLS, A ‘-’ INDICATES THAT THE NUMBER HAS NOT BEEN REPORTED IN THE CORRESPONDING ARTICLE

Dataset	Project	CoCoNuT [11]	CURE [37]	RewardR. [21]	AlphaR. [19]	RAPGEN [39]	SelfAPR [27]	Replit [29]	InstructRepair
Defects4J v1.2	Chart	6	9	5	8	8	7	6	9
	Closure	9	13	15	22	20	19	22	22
	Lang	7	9	7	12	11	10	15	9
	Math	14	18	19	19	17	22	21	25
	Mockito	3	3	2	4	2	3	0	3
	Time	1	1	1	3	1	3	2	1
	Total	40	53	49	68	59	64	66	69
Defects4J v2.0	Cli	-	-	7	5	6	8	6	7
	Codec	-	-	5	6	4	8	6	8
	Collections	-	-	0	0	1	1	1	1
	Compress	-	-	4	1	5	6	2	8
	Csv	-	-	3	1	2	1	3	3
	JacksonCore	-	-	2	3	4	3	3	4
	JacksonDataBind	-	-	9	7	11	8	7	11
	Jsoup	-	-	8	9	10	9	18	11
	Total	-	-	42	35	47	44	46	53
Total		40	53	91	103	106	110	112	122

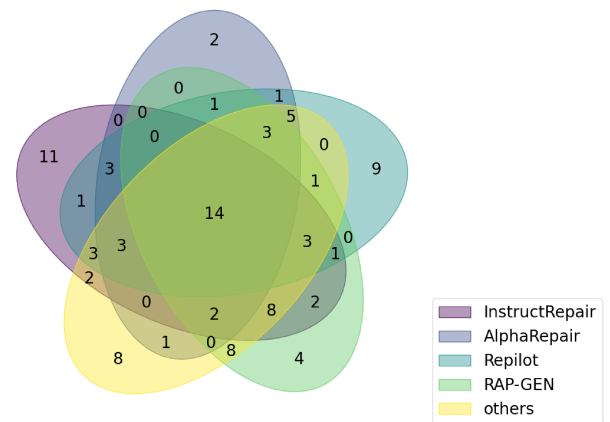
achieves the new SOTA record of successfully repairing 122 bugs in total from Defects4J, which outperforms the best fine-tuning-based approaches (SelfAPR [27]) by 12 (+10.9%) more bugs and prompt-based approach (Replit [29]) by 10 (+8.9%) more bugs. This result demonstrates the superior repair effectiveness of InstructRepair.

Specifically, in Defects4J v1.2, InstructRepair successfully fixes 9, 22, 9, 25, 3, 1 bug(s) for the six real-world projects, three of which are the best-performing (bold in Table I). In terms of Defects4J v2.0, InstructRepair beats out top-performing APRs in six out of eight projects. It’s worth noting that Defects4J v2.0 is widely adopted by SOTAs to deal with existing dataset overfitting issue [78], [83], [84] within Defects4J v1.2, and to examine both their repair effectiveness and generalizability [19], [29]. Our work improves the next best baseline RAP-GEN [39] by 6 (12.8%) more bugs in Defects4J v2.0, indicating that the training of InstructRepair and inference with rich bug information is effective, and our work does not suffer from the generalizability issues faced by other learning-based APRs.

To further demonstrate InstructRepair’s advantage, we analyze the number of unique bugs fixed exclusively by our work. Figures 4a and 4b illustrate that InstructRepair produces a total of 19 unique patches, which all SOTAs fail to generate for Defects4J. In particular, InstructRepair provides the most unique patches for Defects4J v2.0, successfully fixing 11 previous unfixed bugs. After we review these bugs, we find that our work can easily fix those bugs that need to synthesize some extra tokens, which could not be found in buggy code but in test suite code. Listing 1 and 2 are two examples that InstructRepair can uniquely fix. By providing rich bug information, including test suite code, and clear instructions about each element, InstructRepair successfully repairs these bugs while other baselines can hardly fix them due to the lack of certain bug information with instructions. This further affirms the repair effectiveness of our novelty by leveraging the power



(a) for Defects4J v1.2



(b) for Defects4J v2.0

Fig. 4. Correct patch Venn diagrams.

of instruction tuning, prompt tuning and rich bug information in the field of APR, and also indicates that InstructRepair can

TABLE II
PERFORMANCE OVER DIFFERENT BEAM WIDTHS

Beam Width	Identical Patch	Plausible Patch	Total
1	35	33	63(51.6%)
5	50	76	103(88.0%)
10	53	90	117(95.9%)
20	57	95	122(100%)

```
//TEST SUITE CODE
String s = "<p>Hello</p><script>...</script><p>
    There</p>";
Document doc = Jsoup.parse(s);
//BUGGY CODE
else if (StringUtil.in(name, "base", "basefont" ...
    "title")) {
//PATCH
else if (StringUtil.in(name, "base", "basefont" ...
    "title", "script")) {
```

Listing 1. Repair of Jsoup-15

```
//TEST SUITE CODE
assertPrint("var x = '\0';", "var x = '\000'");
//BUGGY CODE
case '\0': sb.append("\0"); break;
//PATCH
case '\0': sb.append("\000"); break;
```

Listing 2. Repair of Closure-65

TABLE III
COMPARISON OF PATCH RANKING EFFECTIVENESS WITH SOTAS OVER DEFECTS4J. IN THE CELLS, A '-' INDICATES THAT THE NUMBER HAS NOT BEEN REPORTED IN THE CORRESPONDING ARTICLE

Approaches	Beam Width (k)	Top-10 / Top-k
CoCoNuT [11]	1000	-
AlphaRepair [19]	5000	-
Repilot [29]	5000	-
CURE [37]	1000	25.7%
RewardRepair [21]	200	32.9%
RAP-GEN [39]	100	37.6%
SelfAPR [27]	50	60.0%
InstructRepair	20	95.9%

be combined with other SOTAs using a collaborative strategy to reach a higher overall repairing performance [85], [86].

B. RQ2 (Patch Ranking Effectiveness): How Does InstructRepair Perform Under Different Beam Widths?

We conduct experiments on beam width settings of InstructRepair under Defects4J. As reported in Table II, setting a relatively small beam width (as 10) results in a total of 117 correct patches, which indicates that 95.9% of patches are ranked in the top-10 by InstructRepair. This result highlights the strong patch ranking effectiveness of our work, demonstrating the practicality of our approach for real-world workflows.

Table III compares patch ranking effectiveness between InstructRepair and SOTAs. Compared with other work that only 25.7% ~ 60.0% of correct patches are ranked in the

top-10, InstructRepair remarkably outperforms all SOTAs. The comparison of APRs' original beam width (given in the second column) also indicates that, attributing to the strong patch ranking effectiveness, InstructRepair requires the fewest candidate patches (only 20 as beam width) to repair a bug among these SOTAs and can handle real-world bugs with ease according to Table I and III.

C. RQ3 (Ablation Study): What Is the Contribution of Each Component Within InstructRepair?

In RQ3, an ablation study is conducted to examine the contribution of individual components within InstructRepair. Table IV presents the results of InstructRepair and its variants under Defects4J.

We first observe that InstructRepair_o, which follows prior fine-tuning-based APRs [11], [37] using fine-tuning without any instructions or test suite code, repairs the fewest bugs, resulting in 50 in total for Defects4J after removing duplicate patches within identical patches and plausible patches. Next, by adding hand-crafted instructions in the training process but without test suite code, InstructRepair_{w/_instruction} can repair 23 more bugs than InstructRepair_o in total, suggesting that instruction tuning remains powerful in the APR field. Afterwards, by only integrating test suite code into the APR workflow, InstructRepair_{w/_test-suite} also outperforms InstructRepair_o, successfully fixing 4 more bugs. This result indicates that simply providing test suites to the patch generator is inherently beneficial. Future APRs, whether based on instruction tuning or other learning techniques, can extract bug-triggering information from test suites as well to supply additional fix-relevant tokens and insights into the root causes of bugs. Furthermore, by combining both hand-crafted instructions and test suite code—without further refining the instruction template, InstructRepair_{w/o_prompt-tuning} outperforms both InstructRepair_{w/_instruction} and InstructRepair_{w/_test-suite}, repairing 6 and 25 more bugs, respectively. This confirms that instruction tuning and test suite code integration complement each other, jointly improving repair performance. Finally, InstructRepair_{full} conduct further automated refining on the hand-crafted instruction template through prompt tuning, achieving the best performance with a total of 103 bugs fixed—4 more than InstructRepair_{w/o_prompt-tuning} and 33 more than InstructRepair_o, demonstrating that prompt tuning further enhances InstructRepair's performance.

We note that InstructRepair_{full} generates 12 more identical patches than InstructRepair_o. This indicates that our work is powerful in syntactic APR training which guides LLMs to generate token-level identical patches. In terms of plausible patch, InstructRepair_{full} produces 10 more patches as well, demonstrating that our work is also powerful in semantic APR training which enables LLMs to produce functional-equivalent patches. While syntactic APR training may bring a potential patch-overfitting problem [21], InstructRepair can mitigate such a problem by instructing LLMs to semantically analyze buggy programs' misbehavior and transfer learned repairing knowledge into real-world bug repairing.

TABLE IV
ABLATION STUDY

Approach	Identical Patch	Plausible Patch	Total
InstructRepair ₀	32	45	70(-32.0%)
InstructRepair _{w/_instruction}	47	64	93(-9.7%)
InstructRepair _{w/_test-suite}	41	49	74(-28.1%)
InstructRepair _{w/o_prompt-tuning}	52	67	99(-3.8%)
InstructRepair _{full}	54	69	103

TABLE V
COMPARISON OF INSTRUCTREPAIR WITH SOTAS OVER BUGSINPY

Approach	Alp. [19]	CoC. [11]	PyT. [7]	Type. [62]	Ins.Re.
Total	5	6	15	26	30

Overall, the ablation study confirms that both instruction tuning and test suite code within InstructRepair are beneficial for the generation of more accurate patches.

D. RQ4 (Generalizability): How Does InstructRepair Perform Across Different Programming Languages or Datasets?

Table V presents the results of InstructRepair against BugsInPy and compares with four SOTAs. InstructRepair exhibits the best performance, successfully repairing 30 bugs (15% more than the best baseline) of BugsInPy, surpassing both cross-language ARPs and Python-only ARPs. This result affirms the generalizability of InstructRepair that can not only achieve promising repairing capability over the most widely used Java benchmark, Defects4J v1.2 and newer yet harder Java benchmark, Defects4J v2.0, but also generalize to the widely adopted real-world open-source APR benchmark, BugsInPy [41] which designed for the broadly applied Python programming language.

VI. THREATS TO VALIDITY

A. Internal Threats to Validity

One internal threat to validity comes from the manual validation of correct patches. To address this, we follow prior work to make all generated patches and our manually verified correct patches publicly available for further assessment.

The second internal threat relates to the experimental settings. Our evaluation of repair effectiveness and patch ranking effectiveness may be influenced by the specific experimental settings (e.g., hyper-parameters, hardware platform). Considering that optimizing perfect hyper-parameters and isolating hardware's influence for LLMs is both expensive and time-consuming [87], we simply aim to propose a novel APR pipeline and do not over-consider which experimental settings will maximize InstructRepair's performance. Nevertheless, InstructRepair still outperforms SOTAs, and its performance can be further improved potentially. To mitigate this threat, we report detailed experimental settings in Section IV-D and our

online repository to avoid such influence on performance and ensure InstructRepair's reproducibility.

B. External Threats to Validity

A threat to external validity relates to our evaluation datasets, where InstructRepair may not generalize to other bug-fix datasets. To mitigate this issue, we choose two of the most widely adopted bug-fix datasets: Defects4J v1.2 and v2.0 [28] for Java and another broadly applied real-world open-source Python benchmark: BugInPy [41] per prior work to demonstrate InstructRepair's generalizability across different evaluation benchmarks.

Another external threat comes from the programming language we choose: Java. We implement InstructRepair on Java because according to [2], [3], and [88], Java is the most broadly applied programming language in modern software systems, and is the most targeted language of mature APR datasets. Besides, in terms of APR research paper distribution on programming language, Java is also the dominant language. Additionally, we implement our tool on another popular Python programming language to demonstrate InstructRepair's cross-language generalization capability. We note that our work can be further extended to various programming languages since the APR framework we propose is general, and can be applied to other bug-fix datasets. Future work can construct instructional bug-fix datasets for other representative programming languages upon InstructRepair.

The third external threat relates to the type of bugs for which InstructRepair is applicable. As in Section IV-B, we primarily focus on single-hunk bugs for evaluation. It has been acknowledged that repairing multi-hunk bugs is an open challenge [3], [15]. Therefore, following all baselines, InstructRepair is mainly designed to handle single-hunk bugs, not multi-hunk bugs. It is interesting to extend InstructRepair on handling multi-hunk bugs in the future. More specifically, future work may design multi-turn repair instruction templates to extend our approach to an agentic pipeline. This allows agentic LLMs to tackle one edit at a time and loop until all test failures are resolved, reducing context overload with which multi-hunk bugs involve. Apart from this, the APR community may prune the unnecessary code context and adapt instruction templates to enumerate multiple edits. Such method requires instructional prompts to be structured in steps like "(1) fix bug in File A, (2) fix bug in File B, ...". We also need multi-hunk fixes including the buggy lines, context, and other bug information for each hunk to teach LLMs to map a multi-part instruction to a multi-part patch.

VII. CONCLUSION

In this paper, we introduced InstructRepair, a novel pipeline for APR that collaboratively leverages instruction tuning and integrates rich bug information. By bridging the gap between prompt-based and fine-tuning-based approaches, InstructRepair harnesses the strengths of both methods while mitigating their limitations, thereby providing a unified framework that improves the repair effectiveness and patch ranking effectiveness of LLMs. The exploitation of bug-triggering test suite information provides additional fix-relevant tokens and insights into the root causes of bugs for LLMs, leading to more accurate fixes. We also use prompt tuning to refine our hand-crafted instruction template with task-specific representations automatically, further boosting InstructRepair's repair effectiveness. Through extensive evaluations on Defects4J v1.2 and v2.0 benchmarks, InstructRepair outperforms Java SOTAs by successfully repairing 122 bugs and generating 19 unique patches. Apart from that, InstructRepair beats Python SOTAs by generating 15% more patches, demonstrating its cross-language generalization capability. Also, under practical circumstances, InstructRepair maintains a competitive repair performance with the most high-quality patches among SOTAs.

APPENDIX A

GLOSSARY OF TERMS AND ABBREVIATIONS

APR

Automated Program Repair. A field focused on automatically fixing software bugs using techniques such as program synthesis, search, and learning.

LLM

Large Language Model. A type of deep learning model trained on vast amounts of textual data to understand and generate human-like language. Used in tasks such as code completion, repair, summarization, and synthesis.

Defects4J

A Java benchmark dataset for evaluating program repair and testing techniques. It consists of real bugs from open-source Java projects, along with test suites and patch information.

BugsInPy

A benchmark dataset for evaluating Python program repair tools. It includes real bugs from open-source projects, along with their test suites and buggy/fixed versions.

Prompting The technique of guiding a language model's output by crafting specific input formats or textual instructions without changing the model's internal parameters.

Fine-Tuning

The process of continuing training on a pre-trained model using task-specific labeled data to adapt it to a particular downstream task.

Prompt-Tuning

A lightweight adaptation method where learnable prompt tokens are trained to steer a frozen language model's behavior on a target task, instead of modifying model parameters.

Instruct-Tuning

A form of supervised fine-tuning where a language model is trained on instruction-following data to improve its ability to follow natural-language prompts across tasks.

Test Suite

A set of test cases designed to check the correctness of a program. In APR, usually used to validate whether a patch fixes the bug without breaking existing functionality.

AST

Abstract Syntax Tree. A tree representation of the abstract syntactic structure of code. Used in APR for analyzing and transforming programs.

Bug's Diagnostics

Information such as test failures, compiler errors, or static analysis reports that can help fix the bug in the program.

Bug's Context

The local code structure (surrounding syntax, control flow, or variable usage) around a buggy location. Useful for guiding the generation or validation of patches.

APPENDIX B

TRAINING CONFIGURATIONS OF INSTRUCTREPAIR

Model and Libraries

We use the CodeT5+ model accessed via the Hugging Face API. Instruction tuning and prompt tuning are implemented using the Transformers library. Abstract syntax tree extraction is performed using javalang.

Tuning Strategy

InstructRepair first performs instruction tuning followed by prompt tuning. This two-stage approach allows the model to first learn general instruction-following behaviors and then refine its task performance via lightweight, soft prompt tuning.

Hyperparameter Settings

For instruction tuning: we use 7 training epochs, a learning rate of 5×10^{-5} , and 4000 warm-up steps. For prompt tuning: we use 3 training epochs, a sequence-to-sequence task objective, 40 tunable soft tokens, and a learning rate of 3×10^{-5} . We set the beam width to 20 and use checkpoint ensembling based on epoch snapshots.

Input and Context Configuration

The token limit for model input is set to 512. This conservative setting helps reduce context size, making the approach more efficient and suitable for practical deployment.

Training Hardware and Runtime

All experiments were conducted on an Ubuntu 20.04.6 server equipped with 4 NVIDIA RTX A6000 GPUs, 2 Intel Xeon Platinum 8380 CPUs, 128GB memory, and 4TB SSD storage. The full training time of InstructRepair is approximately 15 days, and inference time is approximately 4 days. For bug validation, we apply a 5-hour timeout per instance.

REFERENCES

- [1] M. Monperrus, "Automatic software repair: A bibliography," *ACM Comput. Surveys*, vol. 51, no. 1, pp. 1–24, Jan. 2019, doi: [10.1145/3105906](https://doi.org/10.1145/3105906).
- [2] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, "A survey of learning-based automated program repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 2, pp. 1–69, Feb. 2024, doi: [10.1145/3631974](https://doi.org/10.1145/3631974).
- [3] K. Huang et al., "A survey on automated program repair techniques," 2023, *arXiv:2303.18184*.
- [4] D. H. O'Dell, "The debugging mindset: Understanding the psychology of learning strategies leads to effective problem-solving skills," *Queue*, vol. 15, no. 1, pp. 71–90, Feb. 2017, doi: [10.1145/3055301.3068754](https://doi.org/10.1145/3055301.3068754).

- [5] B. Schroeder and G. A. Gibson, "A large-scale study of failures in high-performance computing systems," *IEEE Trans. Dependable Secure Comput.*, vol. 7, no. 4, pp. 337–350, Oct. 2010, doi: [10.1109/TDSC.2009.4](#).
- [6] K. Liu, A. Koyuncu, D. Kim, and T. F. Bissyandé, "TBar: Revisiting template-based automated program repair," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, 2019, pp. 31–42, doi: [10.5281/zenodo.3237378](#).
- [7] W. Oh and H. Oh, "PyTER: Effective program repair for Python type errors," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2022, pp. 922–934, doi: [10.1145/3540250.3549130](#).
- [8] D. Kim, J. Nam, J. Song, and S. Kim, "Automatic patch generation learned from human-written patches," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, May 2013, pp. 802–811, doi: [10.1109/ICSE.2013.6606626](#).
- [9] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 6000–6010. [Online]. Available: <https://dl.acm.org/doi/10.5555/3295222.3295349>
- [10] R. Bavishi, H. Yoshida, and M. R. Prasad, "Phoenix: Automated data-driven synthesis of repairs for static analysis violations," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2019, pp. 613–624, doi: [10.1145/3338906.3338952](#).
- [11] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "CoCoNuT: Combining context-aware neural translation models using ensemble for program repair," in *Proc. 29th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2020, pp. 101–114, doi: [10.1145/3395363.3397369](#).
- [12] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 1430–1442, doi: [10.1109/ICSE48619.2023.00125](#).
- [13] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proc. 6th ACM SIGPLAN Int. Symp. Mach. Program.*, Jun. 2022, pp. 1–10, doi: [10.1145/3520312.3534862](#).
- [14] Z. Zheng et al., "A survey of large language models for code: Evolution, benchmarking, and future trends," 2023, *arXiv:2311.10372*.
- [15] K. Huang et al., "An empirical study on fine-tuning large language models of code for automated program repair," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2023, pp. 1162–1174, doi: [10.1109/ASE56229.2023.00181](#).
- [16] J. Cao, M. Li, M. Wen, and S.-C. Cheung, "A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair," *Automated Softw. Eng.*, vol. 32, no. 1, pp. 1–29, May 2025, doi: [10.1007/s10515-025-00492-x](#).
- [17] S. B. Hossain et al., "A deep dive into large language models for automated bug localization and repair," *Proc. ACM Softw. Eng.*, vol. 1, pp. 1471–1493, Jul. 2024, doi: [10.1145/3660773](#).
- [18] C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for 0.42 each using ChatGPT," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Sep. 2024, pp. 819–831, doi: [10.1145/3650212.3680323](#).
- [19] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2022, pp. 959–971, doi: [10.1145/3540250.3549101](#).
- [20] M. Tufano, C. Watson, G. Bavota, M. D. Penta, M. White, and D. Poshyvanyk, "An empirical study on learning bug-fixing patches in the wild via neural machine translation," *ACM Trans. Softw. Eng. Methodology*, vol. 28, no. 4, pp. 1–29, Sep. 2019, doi: [10.1145/3340544](#).
- [21] H. Ye, M. Martinez, and M. Monperrus, "Neural program repair with execution-based backpropagation," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 1506–1518, doi: [10.1145/3510003.3510222](#).
- [22] J. Zhang, C. Wang, A. Li, W. Wang, T. Li, and Y. Liu, "VulAdvisor: Natural language suggestion generation for software vulnerability repair," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 1932–1944, doi: [10.1145/3691620.3695555](#).
- [23] J. White et al., "A prompt pattern catalog to enhance prompt engineering with ChatGPT," in *Proc. 30th Conf. Pattern Lang. Programs (PLoP)*, 2023, pp. 1–7. [Online]. Available: <https://dl.acm.org/doi/10.5555/3721041.3721046>
- [24] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2339–2356, doi: [10.1109/SP46215.2023.10179324](#).
- [25] K. Beck, *Test Driven Development: By Example*. Reading, MA, USA: Addison-Wesley, 2022. [Online]. Available: <https://dl.acm.org/doi/book/10.5555/579193>
- [26] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Trans. Softw. Eng.*, vol. 42, no. 8, pp. 707–740, Aug. 2016, doi: [10.1109/TSE.2016.2521368](#).
- [27] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, "SelfAPR: Self-supervised program repair with test execution diagnostics," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2022, pp. 1–13, doi: [10.1145/3551349.3556926](#).
- [28] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proc. Int. Symp. Softw. Test. Anal.*, Jul. 2014, pp. 437–440, doi: [10.1145/2610384.2628055](#).
- [29] Y. Wei, C. S. Xia, and L. Zhang, "Copiloting the copilots: Fusing large language models with completion engines for automated program repair," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 172–184, doi: [10.1145/3611643.3616271](#).
- [30] F. Liu et al., "FastFixer: An efficient and effective approach for repairing programming assignments," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 669–680, doi: [10.1145/3691620.3695062](#).
- [31] H. Eladawy, C. Le Goues, and Y. Brun, "Automated program repair, what is it good for? Not absolutely nothing!" in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1–13, doi: [10.1145/3597503.3639095](#).
- [32] Y. Noller, R. Shariffdeen, X. Gao, and A. Roychoudhury, "Trust enhancement issues in program repair," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 2228–2240, doi: [10.1145/3510003.3510040](#).
- [33] J. Wei et al., "Finetuned language models are zero-shot learners," 2021, *arXiv:2109.01652*.
- [34] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [35] A. Grattafiori et al., "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.
- [36] Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, and S. Hoi, "CodeT5+: Open code large language models for code understanding and generation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Dec. 2023, pp. 1069–1088. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.68/>
- [37] N. Jiang, T. Lutellier, and L. Tan, "CURE: Code-aware neural machine translation for automatic program repair," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, May 2021, pp. 1161–1173, doi: [10.1109/ICSE43902.2021.00107](#).
- [38] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2021, pp. 3045–3059. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.243/>
- [39] W. Wang, Y. Wang, S. Joty, and S. C. H. Hoi, "RAP-gen: Retrieval-augmented patch generation with CodeT5 for automatic program repair," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 146–158, doi: [10.1145/3611643.3616256](#).
- [40] K. Liu, A. Koyuncu, T. F. Bissyandé, D. Kim, J. Klein, and Y. Le Traon, "You cannot fix what you cannot find! An investigation of fault localization bias in benchmarking automated program repair systems," in *Proc. 12th IEEE Conf. Softw. Test., Validation Verification (ICST)*, Apr. 2019, pp. 102–113, doi: [10.1109/ICST.2019.00020](#).
- [41] R. Widiasari et al., "BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2020, pp. 1556–1560, doi: [10.1145/3368089.3417943](#).
- [42] D. Fadolalkarim and E. Bertino, "DCAFixer: An automatic tool for bug detection and repair for database Java client applications," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 327–342, Jan. 2025, doi: [10.1109/TDSC.2024.3396667](#).
- [43] M. McFall-Johnsen. (2020). Catastrophic Software Errors Doomed Boeing's Airplanes and Nearly Destroyed Its NASA Spaceship. *Experts Blame the Leadership's 'Lack of Engineering Culture'*. [Online]. Available: <https://www.isvworld.com/search/news/353384>
- [44] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "VulRepair: A T5-based automated software vulnerability repair," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2022, pp. 935–947, doi: [10.1145/3540250.3549098](#).
- [45] D. Yang et al., "TransplantFix: Graph differencing-based code transplantation for automated program repair," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2022, pp. 1–13, doi: [10.1145/3551349.3556893](#).

- [46] A. Afzal, M. Motwani, K. T. Stolee, Y. Brun, and C. Le Goues, "SOSRepair: Expressive semantic search for real-world program repair," *IEEE Trans. Softw. Eng.*, vol. 47, no. 10, pp. 2162–2181, Oct. 2021, doi: [10.1109/TSE.2019.2944914](https://doi.org/10.1109/TSE.2019.2944914).
- [47] G. Zheng et al., "ATR: Template-based repair for alloy specifications," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2022, pp. 666–677, doi: [10.1145/3533767.3534369](https://doi.org/10.1145/3533767.3534369).
- [48] E. Schulte, S. Forrest, and W. Weimer, "Automated program repair through the evolution of assembly code," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, Sep. 2010, pp. 313–316, doi: [10.1145/1858996.1859059](https://doi.org/10.1145/1858996.1859059).
- [49] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest, "Automatically finding patches using genetic programming," in *Proc. IEEE 31st Int. Conf. Softw. Eng.*, May 2009, pp. 364–374, doi: [10.1109/ICSE.2009.5070536](https://doi.org/10.1109/ICSE.2009.5070536).
- [50] C.-P. Wong, P. Santiesteban, C. Kästner, and C. Le Goues, "VarFix: Balancing edit expressiveness and search effectiveness in automated program repair," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2021, pp. 354–366, doi: [10.1145/3468264.3468600](https://doi.org/10.1145/3468264.3468600).
- [51] Y. Ke, K. T. Stolee, C. L. Goues, and Y. Brun, "Repairing programs with semantic code search (T)," in *Proc. 30th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2015, pp. 295–306, doi: [10.1109/ASE.2015.60](https://doi.org/10.1109/ASE.2015.60).
- [52] S. H. Tan and A. Roychoudhury, "Relifix: Automated repair of software regressions," in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, vol. 1, May 2015, pp. 471–482, doi: [10.1109/ICSE.2015.65](https://doi.org/10.1109/ICSE.2015.65).
- [53] Y. Li, S. Wang, and T. N. Nguyen, "DEAR: A novel deep learning-based approach for automated program repair," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 511–523, doi: [10.1145/3510003.3510177](https://doi.org/10.1145/3510003.3510177).
- [54] H. Xu et al., "Large language models for cyber security: A systematic literature review," 2024, *arXiv:2405.04760*.
- [55] X. Zhang et al., "PrivacyAsst: Safeguarding user privacy in tool-using large language model agents," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5242–5258, Nov. 2024, doi: [10.1109/TDSC.2024.3372777](https://doi.org/10.1109/TDSC.2024.3372777).
- [56] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surveys*, vol. 55, no. 9, pp. 1–35, Sep. 2023, doi: [10.1145/3560815](https://doi.org/10.1145/3560815).
- [57] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, Jul. 2018, pp. 328–339. [Online]. Available: <https://aclanthology.org/P18-1031/>
- [58] J. A. Prenner, H. Babii, and R. Robbes, "Can OpenAI's codex fix bugs: An evaluation on quixbugs," in *Proc. IEEE/ACM Int. Workshop Automated Program Repair (APR)*, May 2022, pp. 69–75, doi: [10.1145/3524459.3527351](https://doi.org/10.1145/3524459.3527351).
- [59] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 1482–1494. [Online]. Available: <https://ieeexplore.ieee.org/document/10172803>
- [60] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On hardware security bug code fixes by prompting large language models," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4043–4057, 2024, doi: [10.1109/TIFS.2024.3374558](https://doi.org/10.1109/TIFS.2024.3374558).
- [61] Q. Zhang, C. Fang, T. Zhang, B. Yu, W. Sun, and Z. Chen, "Gamma: Revisiting template-based automated program repair via mask prediction," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2023, pp. 535–547, doi: [10.1109/ASE56229.2023.00063](https://doi.org/10.1109/ASE56229.2023.00063).
- [62] Y. Peng, S. Gao, C. Gao, Y. Huo, and M. Lyu, "Domain knowledge matters: Improving prompts with fix templates for repairing Python type errors," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Feb. 2024, pp. 1–13, doi: [10.1145/3597503.3608132](https://doi.org/10.1145/3597503.3608132).
- [63] J. Gui et al., "A survey on self-supervised learning: Algorithms, applications, and future trends," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9052–9071, Dec. 2024, doi: [10.1109/TPAMI.2024.3415112](https://doi.org/10.1109/TPAMI.2024.3415112).
- [64] Q. Zhang, C. Fang, B. Yu, W. Sun, T. Zhang, and Z. Chen, "Pre-trained model-based automated software vulnerability repair: How far are we?" *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2507–2525, Jul. 2024, doi: [10.1109/TDSC.2023.3308897](https://doi.org/10.1109/TDSC.2023.3308897).
- [65] M. Jin et al., "InferFix: End-to-end program repair with LLMs," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 1646–1656, doi: [10.1145/3611643.3613892](https://doi.org/10.1145/3611643.3613892).
- [66] W. Yuan et al., "CIRCLE: Continual repair across programming languages," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2022, pp. 678–690, doi: [10.1145/3533767.3534219](https://doi.org/10.1145/3533767.3534219).
- [67] B. Wu, S. Liu, R. Feng, X. Xie, J. Siow, and S.-W. Lin, "Enhancing security patch identification by capturing structures in commits," *IEEE Trans. Dependable Secure Comput.*, early access, Jul. 25, 2022, doi: [10.1109/TDSC.2022.3192631](https://doi.org/10.1109/TDSC.2022.3192631).
- [68] X. Zhou, K. Kim, B. Xu, D. Han, and D. Lo, "Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1–13, doi: [10.1145/3597503.3639222](https://doi.org/10.1145/3597503.3639222).
- [69] S. Chaudhary. (2023). *Code Alpaca: An Instruction-Following Llama Model for Code Generation*. [Online]. Available: <https://github.com/sahil280114/codealpaca>
- [70] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2021, pp. 8696–8708. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.685/>
- [71] L. Williams, E. M. Maximilien, and M. Vouk, "Test-driven development as a defect-reduction practice," in *Proc. 14th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Jan. 2003, pp. 34–45, doi: [10.1109/ISSRE.2003.1251029](https://doi.org/10.1109/ISSRE.2003.1251029).
- [72] R. Amankwah, J. Chen, H. Song, and P. K. Kudjo, "Bug detection in Java code: An extensive evaluation of static analysis tools using juliet test suites," *Softw., Pract. Exper.*, vol. 53, no. 5, pp. 1125–1143, May 2023, doi: [10.1002/spe.3181](https://doi.org/10.1002/spe.3181).
- [73] Z. Lai, H. Chen, R. Sun, Y. Zhang, M. Xue, and D. Yuan, "On security weaknesses and vulnerabilities in deep learning systems," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2243–2257, May 2025, doi: [10.1109/TDSC.2024.3482707](https://doi.org/10.1109/TDSC.2024.3482707).
- [74] A. Perez, R. Abreu, and A. Van Deursen, "A test-suite diagnosability metric for spectrum-based fault localization approaches," in *Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. (ICSE)*, May 2017, pp. 654–664, doi: [10.1109/ICSE.2017.66](https://doi.org/10.1109/ICSE.2017.66).
- [75] Y. Tay et al., "UL2: Unifying language learning paradigms," 2022, *arXiv:2205.05131*.
- [76] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "CodeGen2: Lessons for training LLMs on programming and natural languages," 2023, *arXiv:2305.02309*.
- [77] C. Richter and H. Wehrheim, "TSSB-3M: Mining single statement bugs at massive scale," in *Proc. IEEE/ACM 19th Int. Conf. Mining Softw. Repositories (MSR)*, May 2022, pp. 418–422, doi: [10.1145/3524842.3528505](https://doi.org/10.1145/3524842.3528505).
- [78] Y. Ouyang, J. Yang, and L. Zhang, "Benchmarking automated program repair: An extensive study on both real-world and artificial bugs," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Sep. 2024, pp. 440–452, doi: [10.1145/3650212.3652140](https://doi.org/10.1145/3650212.3652140).
- [79] S. M. Jain, "Hugging face," in *Introduction to Transformers for NLP: With the Hugging Face Library and Models to Solve Problems*. Cham, Switzerland: Springer, 2022, pp. 51–67. [Online]. Available: <https://link.springer.com/content/pdf/10.1007/978-1-4842-8844-3.pdf>
- [80] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Process., Syst. Demonstrations*, Oct. 2020, pp. 38–45. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [81] D. S. Kalra and M. Barkeshli, "Why warmup the learning rate? Underlying mechanisms and improvements," in *Proc. 38th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2024, pp. 111760–111801. [Online]. Available: <https://dl.acm.org/doi/10.5555/3737916.3741465>
- [82] Q. Zhu et al., "A syntax-guided edit decoder for neural program repair," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Athens, Greece, Aug. 2021, pp. 341–353, doi: [10.5281/zenodo.5089521](https://doi.org/10.5281/zenodo.5089521).
- [83] E. Ismayilzada, M. M. U. Rahman, D. Kim, and J. Yi, "Poracle: Testing patches under preservation conditions to combat the overfitting problem of program repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 2, pp. 1–39, Feb. 2024, doi: [10.1145/3625293](https://doi.org/10.1145/3625293).
- [84] B. Yang et al., "CREF: An LLM-based conversational software repair framework for programming tutors," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Sep. 2024, pp. 882–894, doi: [10.1145/3650212.3680328](https://doi.org/10.1145/3650212.3680328).
- [85] R. Gharibi, M. H. Sadreddini, and S. M. Fakhrahmad, "T5APR: Empowering automated program repair across languages through checkpoint ensemble," *J. Syst. Softw.*, vol. 214, Aug. 2024, Art. no. 112083, doi: [10.1016/j.jss.2024.112083](https://doi.org/10.1016/j.jss.2024.112083).

- [86] W. Zhong et al., "Practical program repair via preference-based ensemble strategy," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Feb. 2024, pp. 1–13, doi: [10.1145/3597503.3623310](https://doi.org/10.1145/3597503.3623310).
- [87] T. Yu and H. Zhu, "Hyper-parameter optimization: A review of algorithms and applications," 2020, *arXiv:2003.05689*.
- [88] R. Pereira et al., "Ranking programming languages by energy efficiency," *Sci. Comput. Program.*, vol. 205, May 2021, Art. no. 102609, doi: [10.1016/j.scico.2021.102609](https://doi.org/10.1016/j.scico.2021.102609).



Jichunyang Li received the Honours and master's degrees from The University of Western Australia. He is currently an Associate Lecturer with The University of Western Australia. His research interests include large language models and the application of artificial intelligence in educational contexts.



the IoT security, cloud computing security, and privacy preserving.

Anmin Fu (Member, IEEE) received the Ph.D. degree in information security from Xidian University, Xi'an, China, in 2011. He is currently a Professor with Nanjing University of Science and Technology, China. He has published more than 100 technical papers, including international journals and conferences, such as IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE S&P, ACM CCS, NDSS, AsiaCCS, and ACISP. His research interests include



Boyu Kuang received the Ph.D. degree from the School of Computer Science and Engineering, Nanjing University of Science and Technology, in 2023. He is currently an Associate Professor with Nanjing University of Science and Technology. His research interests include the IoT security and the IoV security.



Pengyu Xu received the bachelor's degree from the College of Information Science and Technology, Nanjing Forestry University, Nanjing, China, in 2021. He is currently pursuing the M.Sc. degree with the School of Cyber Science and Engineering, Nanjing University of Science and Technology, Nanjing. His research interests include large language models and automated program repair.



AND LEARNING SYSTEMS.

Yansong Gao (Senior Member, IEEE) received the M.Sc. degree from the University of Electronic Science and Technology of China and the Ph.D. degree from The University of Adelaide, Australia. He is currently a Lecturer at The University of Western Australia. His current research interests include AI security and privacy, hardware security, and system security. He serves as an Associate Editor for IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY and IEEE TRANSACTIONS ON NEURAL NETWORKS